

Principal Component Analysis (PCA)

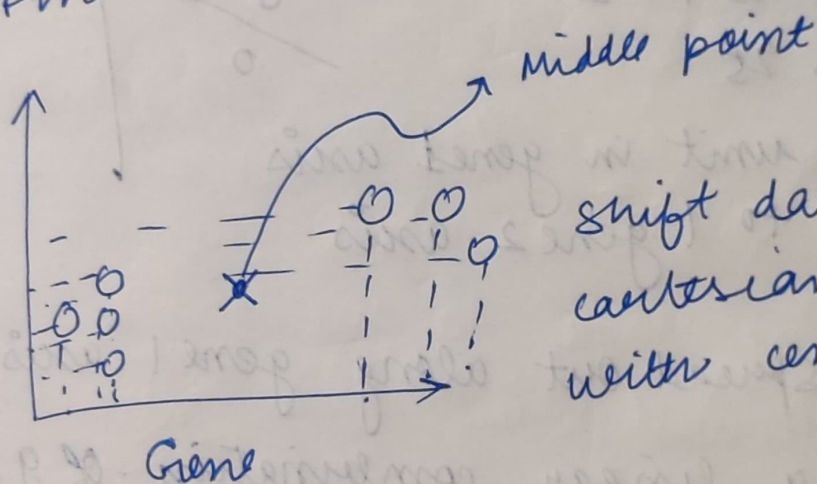
→ The PCA plot converts the correlation among all of the cells in a 2-D graph

→ Plot the data

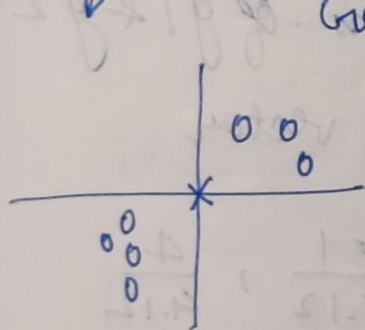
→ calculate average measurement for gene 1

→ Do same for gene 2.

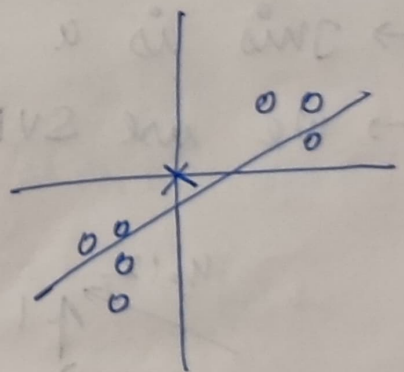
→ Find the center point of data.



shift data to cartesian form with center = (0,0)



→ Try fitting a line to this



→ PCA fits the best fitting line by maximizing the sum of the squared distance from projected points to the origin.

→ We find the sum of squared distances
= SSC (Distance)

And find the largest SS (Distance)

$$\hookrightarrow d_1^2 + d_2^2 + d_3^2 + d_4^2 + d_5^2 \dots$$

→ And that is the PCA Line.

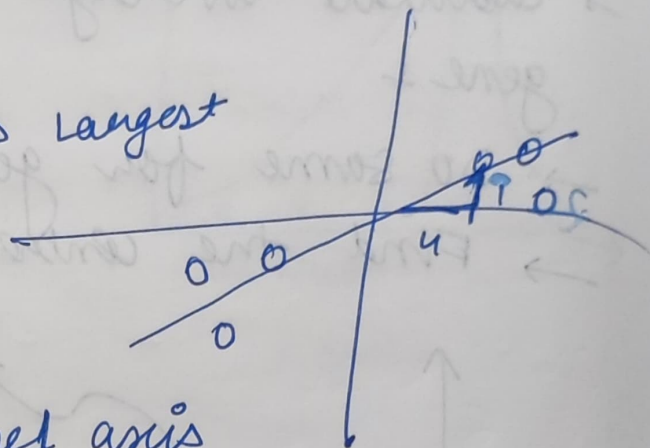
PCA slope

→ This line has largest
SS distance

→ This line is PC 1

→ slope = 0.25

→ Every 4 unit in gene 1 axis
is equal to 1 gene 2 axis



→ Data is spread out along gene 1 axis 4:1

→ This is a linear combination of g_1 & g_2

→ We use SVD to get the unit vector

$$\begin{array}{ccc} \begin{array}{c} 4.12 \\ \nearrow \\ 4 \end{array} & \rightarrow & \frac{4.12}{4.12}, \frac{1}{4.12}, \frac{4}{4.12} \\ & & \rightarrow \text{Gives us 1 unit vector} \\ & & \text{as the eigenvector} \end{array}$$

The ratio 4:1 is called loading score
↓
ratio of gene.

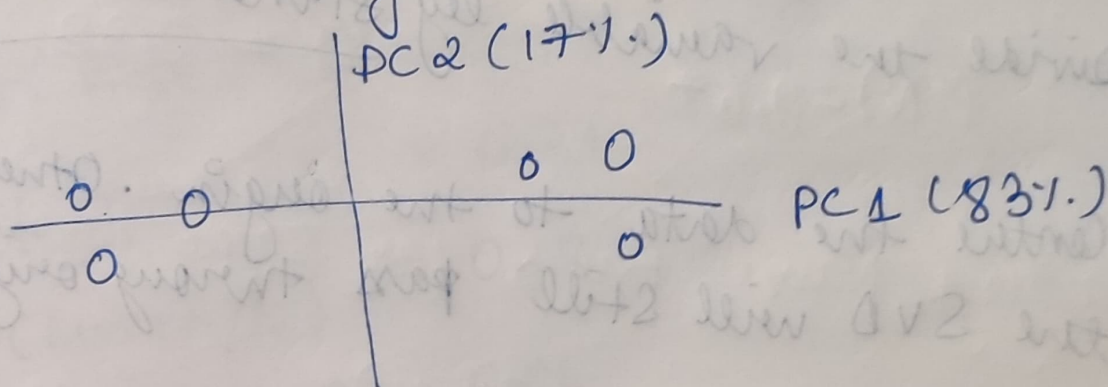
Avg of SS of least fit line the eigen value

$$\frac{SS(\text{distance of PC1})}{n-1} = \text{eigen val}$$

$$\sqrt{SS(\text{distance})} = \text{eigen vector}$$

PC 2 is simply orthogonal to PC 1

→ Eigen values are just the measure of variance



$$\begin{aligned} \text{Assuming } PC1 &= 15 & \Rightarrow & 15/18 = 83\% \\ PC2 &= 3 & \Rightarrow & 3/18 = 17\% \end{aligned}$$

Scree plot tells us about PC's in the data.

→ There is one PCA per var in practice
PC is either number of variables or
number of sample.

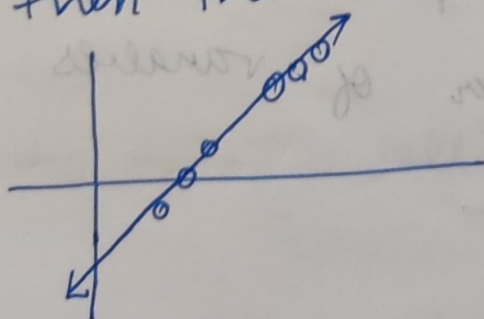
→ When the graph (Scree) is where PC 3 & PC 4 account for substantial amount of variation, then just using the first 2 PC's would not create a very acc rep of data.

PCA TIPS

- ① Make sure the data is on the same scale or normalised
 - Divide the variable by standard dev
- ② Centre the data to the origin. Otherwise the SVD will still pass through origin
- ③ Determining the number of principal component analysis

• If correlation = 1

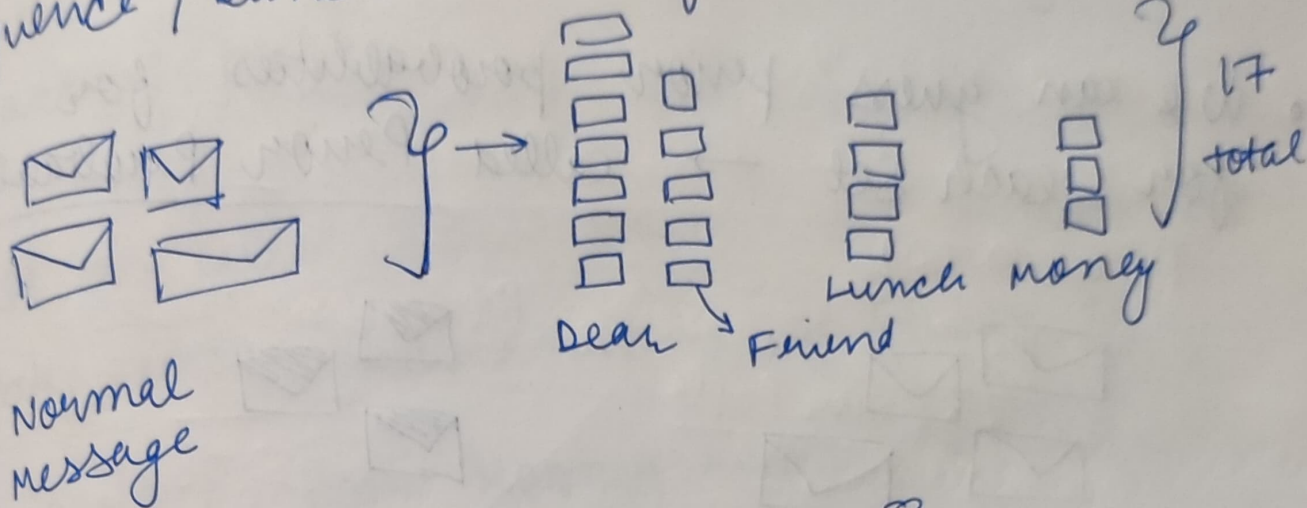
then there is need for just PC 1



make sure eigenval for each PC is > zero.

Naive Bayes

classification and categorisation b/w two things and then predict where each sequence / bunch will fall

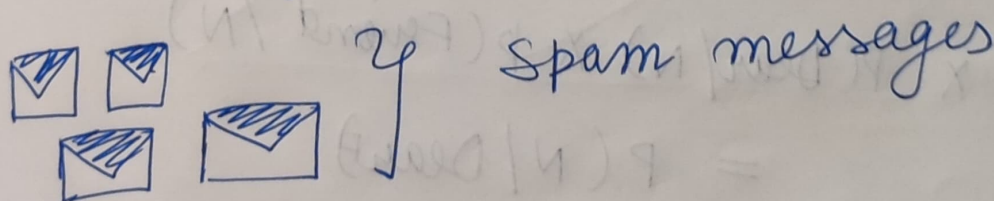


$$P(\text{Dear} | N) = 0.47 = \frac{8}{17}$$

$$P(\text{Friend} | N) = 0.29$$

$$P(\text{Lunch} | N) = 0.18$$

$$P(\text{money} | N) = 0.06$$



$$\rightarrow P(\text{Dear} | S) = 0.29$$

$$P(\text{Friend} | S) = 0.14$$

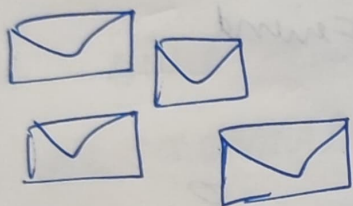
$$P(\text{Lunch} | S) = 0.00$$

$$P(\text{money} | S) = 0.57$$

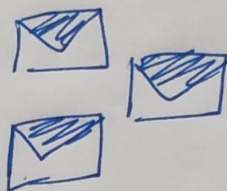
Since the probabilities are discrete and not continuous these are called Likelyhood

→ Likelyhood → name for prob for discrete

• we can guess prior probabilities for each set → called Prior Probability



$$P(N) = 0.67$$



$$P(S) = 0.33$$

Finding Naïve Bayes for some words

DEAR FRIEND

$$P(N) \times P(\text{Dear}/N) \times P(\text{Friend}/N)$$

$$= P(N/\text{Dear})$$

$$= 0.09$$

$$P(S) = 0.33 \times 0.29 \times 0.14$$

$$= 0.001$$

} $0.09 > 0.01$
hence
Normal.

→ sometimes some $P(A)$ might be 0 bcz
it never occurs.
→ But add +1 to all values to get
actual prob.

→ It is called Naive Bayes cuz it does not
take into consideration of order of

$P(N) = \text{Dear Money Money} = 0.8$
 $\text{money money Dear} = 0.8$

order
does not
matter.

Gaussian Bayes

→ Training data is continuous

→ Gaussian distribution → Normal Dist

Loves T_{roll} 2

Does not Love
T_{roll} 2.

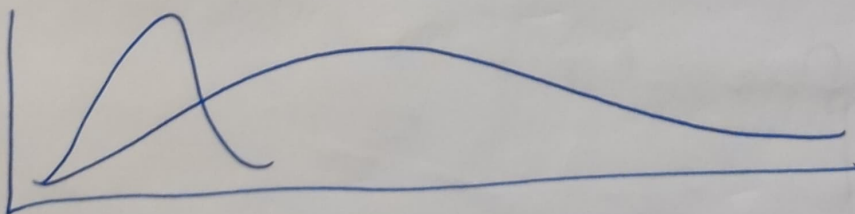
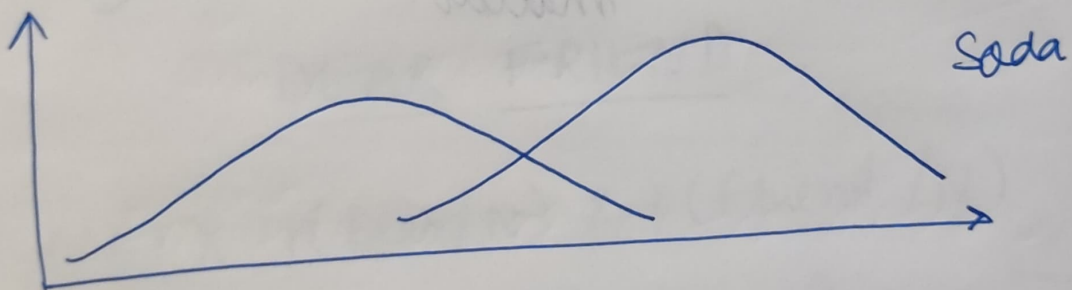
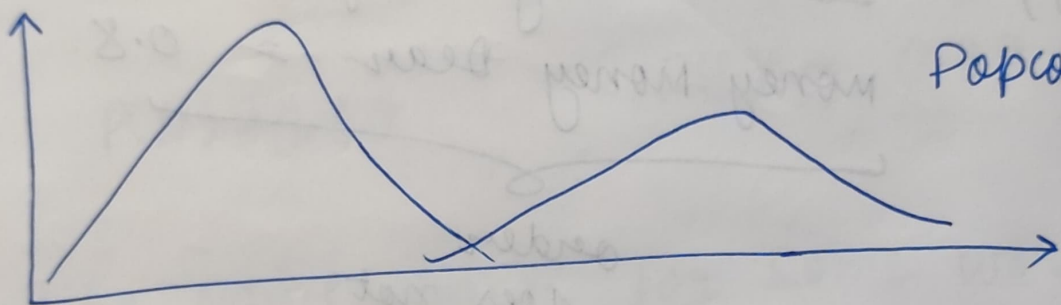
$$P(\text{Loves T}_2)$$

$$= 0.5$$

↳ Prior Prob.

$$P(\text{No Love T}_2)$$

$$= \frac{8}{8+8} = 0.5$$



score for loves troll 2

$$\rightarrow P(\text{loves}) \times L(\text{Popcorn} = 20 / \text{Loves})$$

$$\times L(\text{Soda Pop} / \text{Loves}) \times L(\text{candy} / \text{Loves})$$

$$= 0.5 \times 0.06 \times 0.004 \times \text{small number}$$

↓
this might
lead to underflow

computer can read a number before it gets
close to 0 before it can keep track of number
this can lead to underflow.

→ we solve underflow by taking $\log()$ with
base e

$$\rightarrow \log(0.5) + \log(0.06) + \log(0.004) + \log(\text{small})$$

$$\rightarrow \underline{-124} \Rightarrow \log(\text{Loves troll 2 score})$$

$$\log(\text{Does not love troll 2}) = \underline{-47.66}$$

$$\underline{-47.66} > -124$$

$$\log(L(\text{candy} = 25 / \text{Loves})) = -115$$

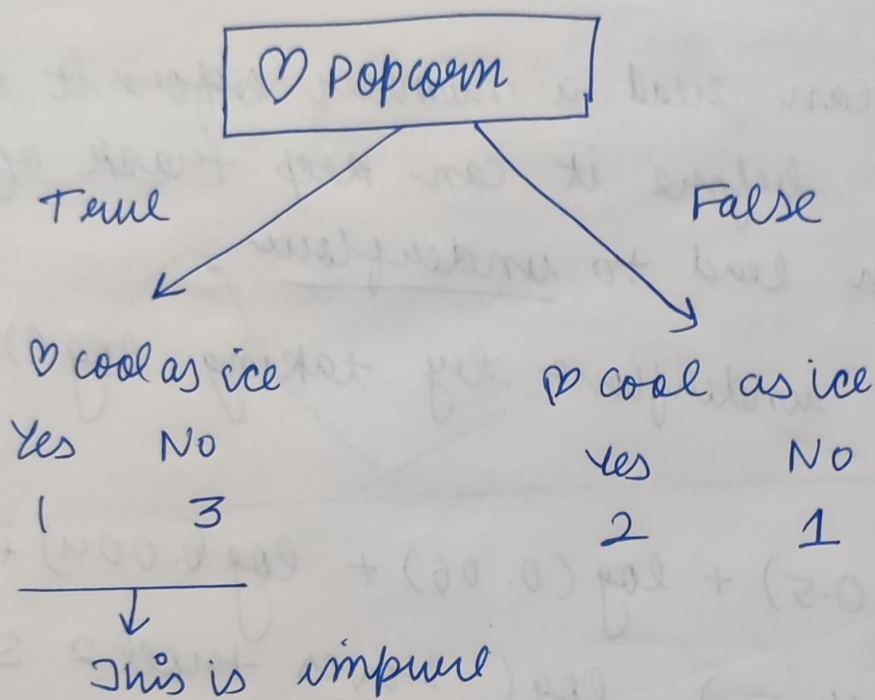
$$\log(L(\text{candy} = 25 / \text{No Love})) = -3.91$$

∴ concludes he does not love troll 2

Tree (Decision and Classification)

→ When a decision tree classifies things into categories → called classification trees.

→ When decision tree predicts numeric values → Regression Tree



Gini Impurity is one of the methods

$$\hookrightarrow \text{Gini Impurity} = 1 - (\text{Prob of Yes})^2 - (\text{Prob of NO})^2$$

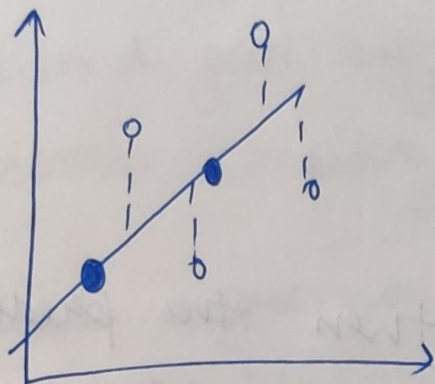
$$\Rightarrow 1 - \left(\frac{1}{1+3}\right)^2 - \left(\frac{3}{1+3}\right)^2$$

$$= 0.375$$

$$G_2 = 0.444$$

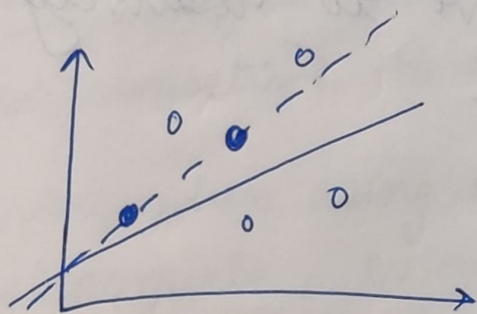
→ The ones with lowest gini impurity are chosen for thresholds. →

Regularisation (L2) Regression (Ridge)



→ The line fits with 0 $SS(R^2)$ but fails with the training data

→ Hence high variance



→ We introduce bias to data to drop in variance

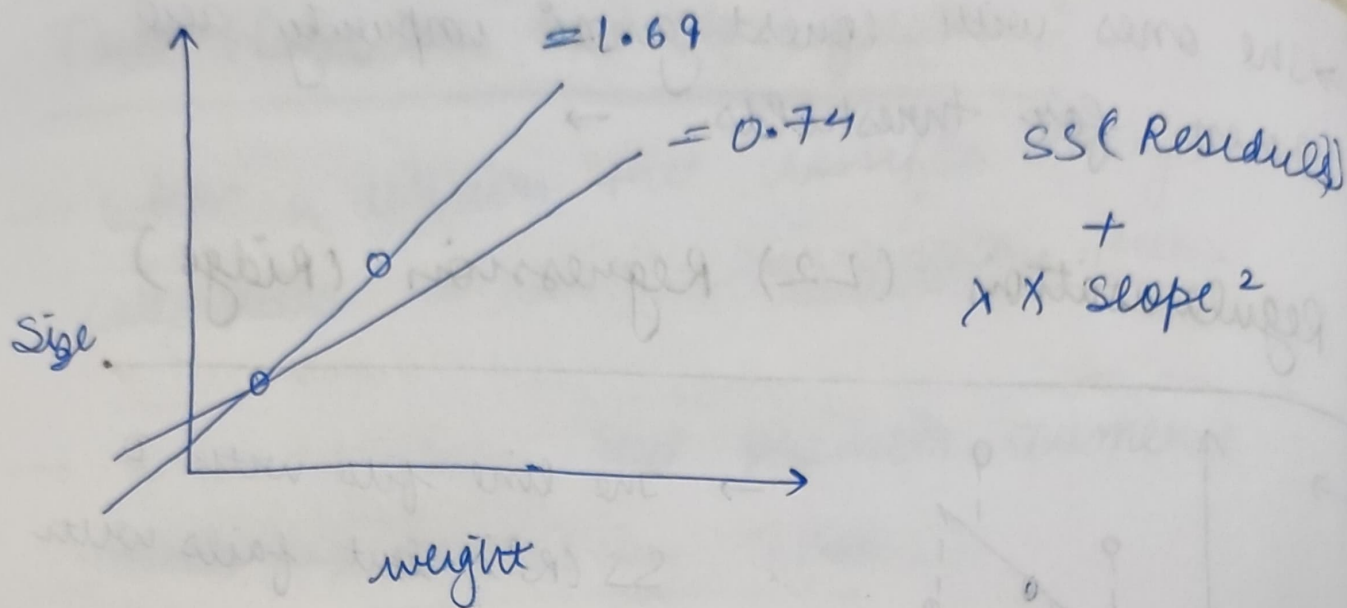
→ For better long prediction

Ridge Regression

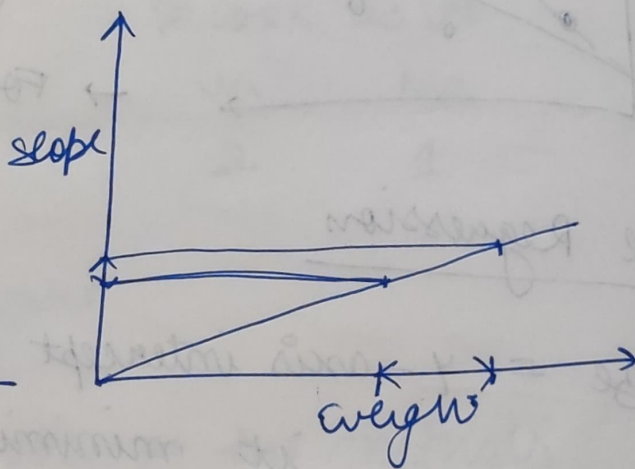
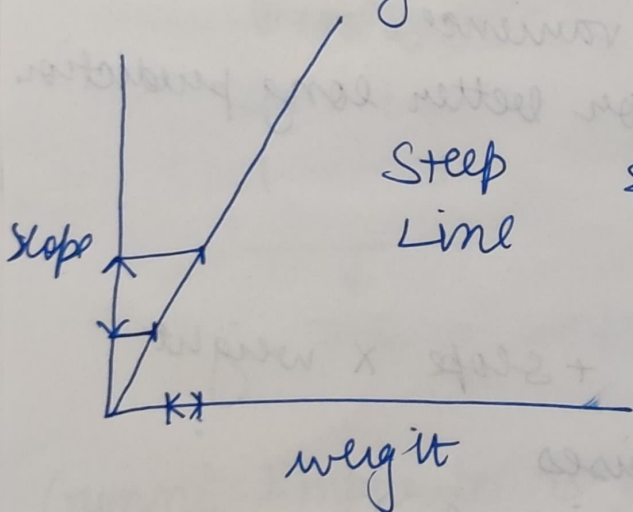
→ Size = y-axis intercept + slope $\times$ weight
it minimises the $SS(\text{Residuals})$

$$+ \lambda \times \frac{\text{slope}^2}{\downarrow}$$

Penalty to traditional method.



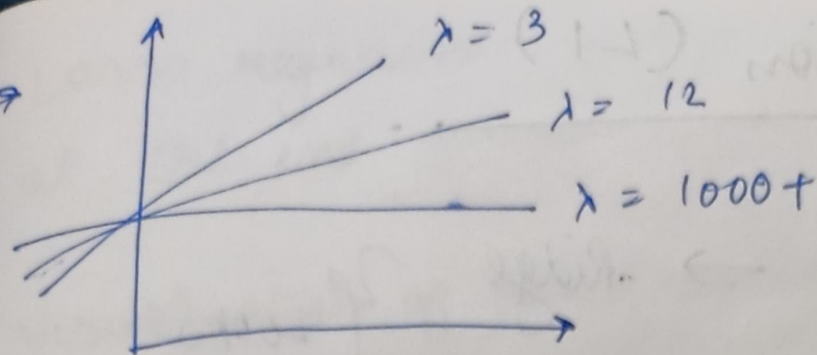
when the slope is steep then the prediction for size is very sensitive to relatively small changes in weight



→ λ can be any value from 0 to $+\infty$

if $\lambda = 0$ then Ridge Reg line minimises SS

→ The larger the λ the slope gets smaller



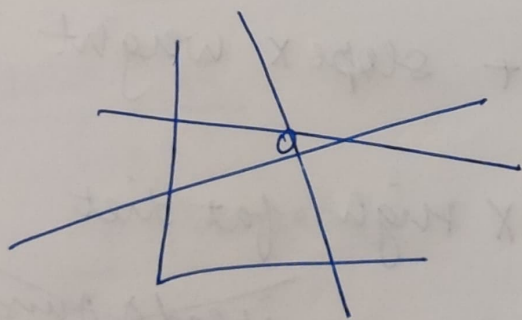
SS
+
 $\lambda \times \text{slope}^2$
↓
this gets bigger

when λ gets bigger the prediction size becomes smaller and smaller

→ Ridge regression helps reduce variance by shrinking parameters and making prediction less sensitive to them.

→ Even when doing ridge regression for high polynomials, everything is squared except y-intercept.

→ we need n data points for n parameters



Any one can work for 2 parameters since all of them have 0 residual.

Lasso Regression (L1)

$$SS(\text{Residual}) \rightarrow \text{Ridge}$$

$$+ \lambda (\text{slope})^2$$

$$SS(\text{Residual}) \rightarrow \text{Lasso}$$

$$+ \lambda |(\text{slope})|$$

} Looks
Similar

→ Ridge regression can shrink the slope close to 0 while Lasso can go upto 0

→ The extra parameters go away in Lasso

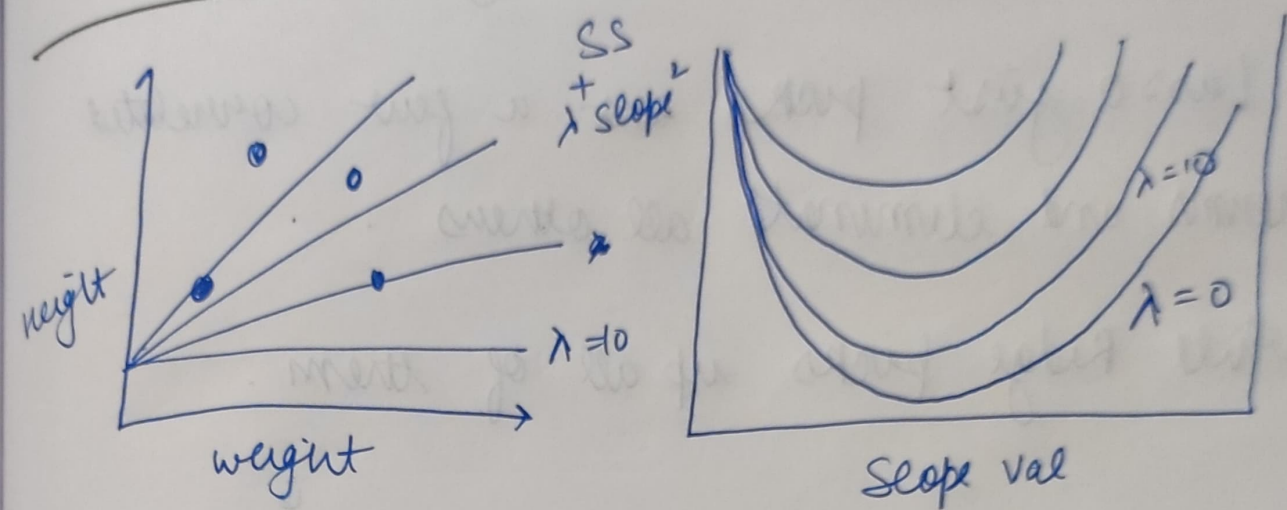
$$\text{Size} = y\text{-intercept} + \text{slope} \times \text{weight}$$

$$+ \text{diet diff} \times \text{high fat diet}$$

$$+ \text{ast} \text{ ~~ast~~ } + \text{anispleed} \text{ ~~anispleed~~ }$$

→ Lasso regression makes equations simple at the end of day

Visualisation of Lasso & Ridge



$$\text{Sum of Sq Residuals} + \lambda \text{ Slope}^2$$

Elastic Regression

- combination of Lasso & Ridge Regression.
- Used when parameters are correlated a lot
- when we have tons of more variables

elastic Net

↳

$$\lambda_1 \times |\text{variable}_1| + \dots + |\text{variable}_x|$$

$\overline{L1}$

+

$$\lambda_2 (\text{variable}_1)^2 + \dots + (\text{variable}_x)^2$$

$\overline{L2}$

when $\lambda_1 > 0$ & $\lambda_2 > 0$

then we get a hybrid elastic net regression

↳ when parameters have high correlation

→ Lasso just picks ~~one~~ a few correlated terms and eliminates all others.

→ while Ridge picks up all of them.